

แนวข้อสอบฝึกซ้อม

หมายเหตุสำคัญ: เอกสารนี้จัดทำขึ้นเพื่อฝึกซ้อมส่วนตัว โดยอ้างอิงจาก "ขอบเขตเนื้อหา" ที่ประกาศไว้ในหนังสือแจ้งรายชื่อผู้มีสิทธิ์สอบ (work_test.pdf) เท่านั้น **ไม่ใช่ข้อสอบจริงที่รั่วไหลออกมา** — เป็นคำถามจำลองที่ครอบคลุมหัวข้อ PHP, SQL, HTML Form (ข้อเขียน) และ PHP+MySQL+Bootstrap+JavaScript/AJAX (ปฏิบัติ) ตามที่ระบุไว้ในประกาศ

ขอบเขตสอบจริง: 15 พ.ค. 2569 — เข้า (ข้อเขียน): PHP / SQL / HTML Form | ปลาย (ปฏิบัติ): Web Application ด้วย PHP+MySQL+Bootstrap+JS/jQuery/AJAX

สารบัญ

ส่วนที่ 1: ข้อสอบข้อเขียน — หมวด PHP (20 ข้อ)

ส่วนที่ 2: ข้อสอบข้อเขียน — หมวด SQL (18 ข้อ)

ส่วนที่ 3: ข้อสอบข้อเขียน — หมวด HTML Form (12 ข้อ)

ส่วนที่ 4: ข้อสอบปฏิบัติ — โจทย์เขียนโปรแกรม (4 โจทย์)

ส่วนที่ 5: เฉลย/แนวคำตอบ

ส่วนที่ 1: หมวด PHP (20 ข้อ)

1. (ปรนัย) ตัวแปรในภาษา PHP ต้องขึ้นต้นด้วยสัญลักษณ์ใด

- ก. #
- ข. \$
- ค. &
- ง. @

2. (ปรนัย) ตัวแปรใดที่รับค่าที่ฟอร์มส่งมาด้วย `method="post"`

- ก. \$_GET
- ข. \$_POST
- ค. \$_SESSION
- ง. \$_FILES

3. (ปรนัย) ฟังก์ชันใดใช้เชื่อมต่อฐานข้อมูล MySQL แบบ procedural style

- ก. mysql_open()
- ข. mysqli_connect()
- ค. db_connect()
- ง. connect_mysql()

4. (ปรนัย) คำสั่งใดใช้รวมไฟล์ PHP อื่นเข้ามา และจะหยุดการทำงานทันทีหากไม่พบไฟล์

- ก. include
- ข. include_once
- ค. require
- ง. import

5. (ปรนัย) ฟังก์ชันใดใช้ตรวจสอบว่าตัวแปรมีค่าว่างหรือไม่มีค่าหรือไม่

- ก. isset()
- ข. empty()
- ค. null()
- ง. is_null_value()

6. (อัตนัย) จงอธิบายความแตกต่างระหว่าง `$_GET` , `$_POST` และ `$_REQUEST` พร้อมยกตัวอย่างสถานการณ์ที่ควรเลือกใช้แต่ละแบบ

7. (อัตนัย) Session กับ Cookie ต่างกันอย่างไร และเพราะเหตุใดระบบสื่อออนไลน์ส่วนใหญ่จึงนิยมใช้ Session

8. (เขียนโค้ด) จงเขียนโค้ด PHP เชื่อมต่อฐานข้อมูล MySQL ชื่อ `school_db` โดยใช้ mysqli (host=localhost, user=root, password ว่าง)

9. (เขียนโค้ด) จงเขียนคำสั่ง PHP วนลูปพิมพ์เลขคู่ตั้งแต่ 2 ถึง 20 โดยใช้ `for`

10. (ทำนายผลลัพธ์) จงบอกผลลัพธ์ของโค้ดต่อไปนี้

```
<?php
$arr = [3, 7, 1, 9];
foreach ($arr as $i => $v) {
    if ($v > 5) {
        echo "index $i = $v ";
    }
}
?>
```

11. (อัตรณ์) จงอธิบายวิธีป้องกัน SQL Injection เมื่อรับค่าจากฟอร์มมาสร้าง Query ใน PHP (แนวคำตอบที่คาดหวัง: prepared statement / parameter binding)

12. (เขียนโค้ด) จงเขียนฟังก์ชัน PHP ชื่อ `sumTwo($a, $b)` ที่รับพารามิเตอร์ 2 ตัว แล้วคืนค่าผลรวม

13. (ปรณัย) `$_SERVER['REQUEST_METHOD']` ใช้เพื่อวัตถุประสงค์ใด

- ก. ตรวจสอบชื่อไฟล์ปัจจุบัน
- ข. ตรวจสอบว่าฟอร์มถูกส่งมาแบบ GET หรือ POST
- ค. ตรวจสอบเวอร์ชัน PHP
- ง. ตรวจสอบ IP ของเซิร์ฟเวอร์

14. (อัตรณ์) จงอธิบายแนวคิด OOP เบื้องต้นใน PHP ได้แก่ class, object, property และ method พร้อมยกตัวอย่างโค้ดสั้นๆ

15. (เขียนโค้ด) จงเขียนโค้ด PHP รับค่าจาก input ชื่อ `username` ที่ส่งมาแบบ POST แล้ว echo ค่านั้นออกมาอย่างปลอดภัย (ป้องกัน XSS)

16. (ปรณัย) ฟังก์ชันใดใช้แปลงอักขระพิเศษ เช่น `<script>` ให้กลายเป็นข้อความธรรมดาก่อนแสดงผล

- ก. `strip_tags()`
- ข. `htmlspecialchars()`
- ค. `trim()`
- ง. `filter_input()`

17. (อัตรณ์) Array แบบ Associative Array ใน PHP คืออะไร ต่างจาก Indexed Array อย่างไร ยกตัวอย่างประกอบ

18. (เขียนโค้ด) จงเขียนโค้ด PHP ตรวจสอบว่า input ชื่อ `age` ที่รับมาเป็นตัวเลขหรือไม่ ถ้าไม่ใช่ให้แสดงข้อความ "กรุณากรอกตัวเลข"

19. (ปรนัย) คำสั่งใดใช้ปิดการเชื่อมต่อฐานข้อมูล MySQL ใน mysqli

- ก. mysqli_close(\$conn)
- ข. mysqli_end(\$conn)
- ค. mysqli_disconnect(\$conn)
- ง. mysqli_stop(\$conn)

20. (อัตนัย) จงอธิบายขั้นตอนการทำงานของระบบ Login ด้วย PHP + MySQL ตั้งแต่ผู้ใช้กรอกฟอร์มจนถึงการอนุญาตเข้าใช้งาน

ส่วนที่ 2: หมวด SQL (18 ข้อ)

1. (ปรนัย) คำสั่งใดใช้ดึงข้อมูลทั้งหมดจากตาราง `students`

- ก. GET * FROM students
- ข. SELECT * FROM students
- ค. FETCH ALL students
- ง. SHOW students

2. (เขียนคำสั่ง) จงเขียนคำสั่ง SQL เพิ่มข้อมูลนักเรียนใหม่ลงในตาราง `students (id, name, age)` ชื่อ "สมชาย" อายุ 15

3. (อัตนัย) JOIN คืออะไร มีกี่ประเภทหลัก จงอธิบาย INNER JOIN และ LEFT JOIN พร้อมความแตกต่าง

4. (เขียนคำสั่ง) จงเขียนคำสั่ง SQL ดึงชื่อนักเรียนที่มีอายุมากกว่า 15 ปี โดยเรียงจากอายุมากไปน้อย

5. (อัตนัย) Primary Key กับ Foreign Key ต่างกันอย่างไร

6. (อัตนัย) `GROUP BY` ใช้ทำอะไร จงยกตัวอย่างร่วมกับฟังก์ชัน `COUNT()`

7. (เขียนคำสั่ง) จงเขียนคำสั่งลบข้อมูลนักเรียนที่มี `id = 5` ออกจากตาราง `students`

8. (เขียนคำสั่ง) จงเขียนคำสั่งแก้ไขชื่อนักเรียนที่ `id = 3` เป็น "สมหญิง"

9. (อัตนัย) `HAVING` ต่างจาก `WHERE` อย่างไร

10. (ปรนัย) คำสั่งใดใช้สร้างตารางใหม่ในฐานข้อมูล

- ก. NEW TABLE
- ข. CREATE TABLE
- ค. MAKE TABLE
- ง. ADD TABLE

11. (เขียนคำสั่ง) จงเขียนคำสั่งสร้างตาราง `students` ประกอบด้วย `id` (INT, Primary Key, Auto Increment), `name` (VARCHAR 100), `age` (INT), `class` (VARCHAR 20)

12. (อัตรณ์) UNIQUE constraint คืออะไร ใช้เมื่อไหร่

13. (เขียนคำสั่ง) มีตาราง `students (id, name)` และ `scores (student_id, subject, score)` จงเขียนคำสั่ง SQL แบบ JOIN เพื่อดึงชื่อนักเรียนพร้อมคะแนนของแต่ละวิชา

14. (อัตรณ์) `ALTER TABLE` ใช้ทำอะไร จงยกตัวอย่างคำสั่งเพิ่มคอลัมน์ `email` เข้าไปในตาราง `students`

15. (อัตรณ์) ความแตกต่างระหว่าง `CHAR` กับ `VARCHAR` คืออะไร

16. (อัตรณ์) Normalization คืออะไร เพราะเหตุใดการออกแบบฐานข้อมูลจึงต้องคำนึงถึงเรื่องนี้

17. (เขียนคำสั่ง) จงเขียนคำสั่ง SQL นับจำนวนนักเรียนทั้งหมดในแต่ละห้อง (class) จากตาราง `students`

18. (ปรณัย) คำสั่งใดใช้ป้องกันไม่ให้ค่าในคอลัมน์เป็นค่าว่าง (NULL)

- ก. NOT NULL
- ข. NO EMPTY
- ค. REQUIRED
- ง. FILLED

ส่วนที่ 3: หมวด HTML Form (12 ข้อ)

1. (ปรนัย) Tag หลักที่ใช้สร้างฟอร์มรับข้อมูลในหน้าเว็บคือ

- ก. <input>
- ข. <form>
- ค. <field>
- ง. <data>

2. (อัตนัย) attribute `action` และ `method` ใน tag `<form>` ใช้ทำหน้าที่ใด

3. (ปรนัย) input type ใดใช้สำหรับรับรหัสผ่านที่ไม่แสดงตัวอักษรบนหน้าจอ

- ก. text
- ข. hidden
- ค. password
- ง. secret

4. (เขียนโค้ด) จงเขียนโค้ด HTML สร้างฟอร์มรับข้อมูล ชื่อ-นามสกุล และอีเมล พร้อมปุ่ม submit โดยส่งข้อมูลแบบ POST ไปยัง `save.php`

5. (อัตนัย) tag `<select>` และ `<option>` ใช้สร้างองค์ประกอบใด จงยกตัวอย่างสร้าง dropdown ให้เลือกกระดืบชั้น (ม.1-ม.6)

6. (ปรนัย) attribute `required` ใน input ทำหน้าที่ใด

- ก. ทำให้ input เป็นสีแดง
- ข. บังคับให้ผู้ใช้กรอกข้อมูลก่อน submit
- ค. ทำให้ input ซ่อนอยู่
- ง. จำกัดจำนวนตัวอักษร

7. (อัตนัย) `<textarea>` ต่างจาก `<input type="text">` อย่างไร ควรใช้กรณีใด

8. (อัตนัย) checkbox กับ radio button ต่างกันอย่างไร ใช้ในสถานการณ์แบบไหน

9. (อัตร้อย) `<label>` กับ attribute `for` เชื่อมโยงกับ input อย่างไร และเหตุใดจึงสำคัญด้าน accessibility

10. (เขียนโค้ด) จงออกแบบฟอร์มลงทะเบียนนักเรียนโดยใช้ Bootstrap class เช่น `form-group`, `form-control`, `btn btn-primary` ให้รองรับการแสดงผลแบบ Responsive

11. (ปรนัย) การส่งฟอร์มแบบ GET จะปรากฏข้อมูลที่ส่งไปตรงส่วนใด

- ก. ใน Cookie
- ข. ใน URL
- ค. ใน Header เท่านั้นและมองไม่เห็น
- ง. ไม่ปรากฏที่ใดเลย

12. (อัตร้อย) เหตุใดจึงไม่ควรใช้ `method="get"` กับฟอร์มที่มีข้อมูลรหัสผ่านหรือข้อมูลอ่อนไหว

ส่วนที่ 4: ข้อสอบปฏิบัติ — โจทย์เขียนโปรแกรม (4 โจทย์)

จำลองจากขอบเขตภาคปฏิบัติ: PHP (back-end) + MySQL (database) + HTML/CSS/Bootstrap (front-end, responsive) + JavaScript/jQuery + AJAX (client-side)

โจทย์ที่ 1: ระบบเพิ่ม/แสดงรายชื่อนักเรียน (CRUD เบื้องต้น)

- สร้างตาราง MySQL ชื่อ `students` ประกอบด้วย id, name, age, class
- สร้างฟอร์ม HTML+Bootstrap รับข้อมูลนักเรียน (ชื่อ, อายุ, ชั้นเรียน)
- เขียน PHP บันทึกข้อมูลลง MySQL เมื่อกด submit
- แสดงรายชื่อนักเรียนทั้งหมดในรูปแบบตาราง Bootstrap (responsive) ด้านล่างฟอร์ม

แนวทางการทำ: ไฟล์เดียวกัน (เช่น `students.php`) สามารถทั้งแสดงฟอร์มและรับข้อมูลได้ โดยใช้รูปแบบ `if ($_SERVER['REQUEST_METHOD'] == 'POST')` { /* รับค่า \$_POST แล้วเขียน INSERT INTO students ... */ } วางไว้บนสุดของไฟล์ก่อนส่วนแสดง HTML จากนั้นค่อย query `SELECT * FROM students` เพื่อวนลูปแสดงผลด้วย `foreach` ในตาราง Bootstrap (`<table class="table table-striped table-responsive">`) ด้านล่างฟอร์ม การ reload หน้าหลัง submit จะทำให้เห็นรายชื่อที่เพิ่งเพิ่มไปปรากฏในตารางทันที

โจทย์ที่ 2: ค้นหานักเรียนแบบ Real-time ด้วย AJAX

- สร้างช่อง input ค้นหาชื่อนักเรียน
- ใช้ jQuery ดักจับ event `keyup` แล้วส่งคำค้นหาไปยัง PHP ผ่าน AJAX (ไม่ reload หน้า)
- PHP รับคำค้นหา query ฐานข้อมูลด้วย `LIKE` แล้วส่งผลลัพธ์กลับเป็น HTML หรือ JSON
- แสดงผลลัพธ์อัปเดตในตารางทันที

แนวทางการทำ: แยกเป็น 2 ไฟล์ — ไฟล์หลัก (`index.php`) มี input ค้นหาและ `<div id="result">` วางไว้รอแสดงผล กับไฟล์ย่อย (`search.php`) ที่รับคำค้นหาแล้ว query `SELECT * FROM students WHERE name LIKE '%คำค้น%'` (ใช้ prepared statement เพื่อความปลอดภัย) แล้ว echo แก่ตารางออกมา
ฝั่ง jQuery: `$('#searchBox').on('keyup', function(){ $.ajax({ url:'search.php', method:'POST', data:{keyword:(this).val()}, success:function(res){ $('#result').html(res); } }); });` จุดสำคัญคือ `success` callback จะนำ HTML ที่ PHP ส่งกลับมาใส่ใน div ผลลัพธ์ทันทีโดยไม่ต้อง reload หน้าทั้งหน้า

โจทย์ที่ 3: ลบข้อมูลนักเรียนด้วย jQuery AJAX พร้อมยืนยันก่อนลบ

- เพิ่มปุ่ม "ลบ" ในแต่ละแถวของตารางนักเรียน
- เมื่อกดปุ่ม ให้แสดงกล่องยืนยัน (confirm) ก่อนลบจริง
- ใช้ AJAX ส่ง id ไปยัง PHP เพื่อลบข้อมูลออกจาก MySQL โดยไม่ reload หน้า
- ลบแถวนั้นออกจากตารางบนหน้าเว็บทันทีเมื่อ AJAX สำเร็จ

แนวทางการทำ: ปุ่มลบแต่ละแถวควรฝัง id ของนักเรียนไว้ เช่น `<button class="btn-delete" data-id="<?=$row['id'] ?>">ลบ</button>` ฝัง jQuery ดักจับ click: `$('.btn-delete').click(function(){ if(confirm('ยืนยันการลบ?')){ let id = $(this).data('id'); let btn = $(this); $.post('delete.php', {id:id}, function(){ btn.closest('tr').remove(); }); } });` ไฟล์ delete.php ฝัง PHP รับ `$_POST['id']` แล้วรัน `DELETE FROM students WHERE id = ?` ด้วย prepared statement จุดสำคัญคือการใช้ `confirm()` ก่อนยิง AJAX เพื่อป้องกันการลบโดยไม่ตั้งใจ และ `.closest('tr').remove()` เพื่อลบแถวออกจากหน้าจอทันทีโดยไม่ต้องดึงข้อมูลใหม่ทั้งตาราง

โจทย์ที่ 4: แก้ไขข้อมูลนักเรียน (Edit) แบบ Modal + AJAX

- เมื่อกดปุ่ม "แก้ไข" ให้เปิด Bootstrap Modal พร้อมดึงข้อมูลเดิมของนักเรียนคนนั้นมาแสดงในฟอร์ม (ผ่าน AJAX)
- ผู้ใช้แก้ไขข้อมูลในฟอร์มแล้วกด "บันทึก"
- ใช้ AJAX ส่งข้อมูลใหม่ไปอัปเดตใน MySQL ผ่าน PHP
- อัปเดตข้อมูลในตารางบนหน้าเว็บให้ตรงกับข้อมูลใหม่ทันทีโดยไม่ reload หน้า

แนวทางการทำ: ขั้นตอนแบ่งเป็น 3 ช่วง (1) กดปุ่มแก้ไข → jQuery เรียก `$.get('get_student.php', {id:id}, function(data){ /* ดึงค่าใน input ของ modal แล้วเปิด modal */ })` โดย get_student.php คืนค่าข้อมูลนักเรียนเป็น JSON ด้วย `echo json_encode($row);` (2) ผู้ใช้แก้ไขค่าในฟอร์มที่อยู่ใน Bootstrap Modal (`<div class="modal">`) แล้วกดบันทึก (3) jQuery ส่งค่าฟอร์มทั้งหมดไปที่ update.php ผ่าน AJAX ซึ่งรัน `UPDATE students SET name=?, age=?, class=? WHERE id=?` เมื่อสำเร็จ ให้ใช้ jQuery อัปเดตข้อความในแถวตารางที่ตรงกัน (เช่น `$('#row-' + id + ' .col-name').text(newName)`) และปิด modal โดยไม่ reload หน้า

ส่วนที่ 5: เฉลย / แนวคำตอบ

เฉลยหมวด PHP

1. ตอบ ข. \$

ตัวแปรทุกตัวใน PHP ต้องขึ้นต้นด้วย `$` เสมอ เช่น `$name`, `$age` ตามด้วยตัวอักษรหรือ underscore (ห้ามขึ้นต้นด้วยตัวเลข) ตัวเลือกอื่นเป็นสัญลักษณ์ที่ใช้ในบริบทอื่น เช่น `#` ใช้ทำ comment แบบบรรทัดเดียว, `&` ใช้ในการอ้างอิงตัวแปร (reference), `@` ใช้ระงับ error message

2. ตอบ ข. \$_POST

เมื่อฟอร์มถูกส่งด้วย `method="post"` ค่าที่กรอกจะถูกส่งไปใน body ของ HTTP request และ PHP จะรับค่าผ่าน superglobal array `$_POST` เท่านั้น (ถ้าฟอร์มใช้ `method="get"` ต้องใช้ `$_GET` แทน) ส่วน `$_REQUEST` ก็รับได้ทั้งสองแบบแต่ไม่นิยมใช้เพราะไม่ชัดเจนว่าค่ามาจากไหน

3. ตอบ ข. mysqli_connect()

เป็นฟังก์ชันแบบ procedural style ที่ใช้เปิดการเชื่อมต่อกับฐานข้อมูล MySQL รับพารามิเตอร์ (host, username, password, database) เช่น `mysqli_connect("localhost", "root", "", "school_db")` ตัวเลือกอื่นในโจทย์ไม่ใช่ฟังก์ชันจริงของ PHP

4. ตอบ ค. require

`include` และ `require` ทำหน้าที่คล้ายกันคือดึงโค้ดจากไฟล์อื่นมารวมกับไฟล์ปัจจุบัน แต่ต่างกันตรงพฤติกรรมเมื่อหาไฟล์ไม่เจอ — `include` จะแค่แสดง Warning แล้วทำงานต่อไป ส่วน `require` จะแสดง Fatal Error แล้วหยุดการทำงานทั้งหมดทันที จึงควรใช้ `require` กับไฟล์ที่จำเป็นต่อการทำงานของระบบ เช่น ไฟล์เชื่อมต่อฐานข้อมูล

5. ตอบ ข. empty()

`empty($var)` คืนค่า true เมื่อตัวแปรไม่มีค่า, มีค่าว่าง (""), เป็น 0, หรือเป็น null ส่วน `isset($var)` ตรวจสอบว่าตัวแปรถูกประกาศและมีค่าไม่ใช่ null หรือไม่ (แม้ค่าจะเป็น "" หรือ 0 ก็ยังถือว่า `isset` เป็น true) มักสับสนกันในข้อสอบ ต้องจำให้แม่นยำว่า `empty()` เข้มงวดกว่า

6.

\$_GET: ส่งค่าแนบไปกับ URL (เช่น `page.php?id=5`) มองเห็นได้ชัดเจน เหมาะกับข้อมูลที่ไม่อ่อนไหวและต้องการแชร์ลิงก์ได้ เช่น การค้นหา, การแบ่งหน้า (pagination), การส่ง id ของสินค้า

\$_POST: ส่งค่าซ่อนอยู่ใน body ของ HTTP request ไม่ปรากฏใน URL รองรับข้อมูลปริมาณมากกว่า เหมาะกับข้อมูลอ่อนไหวหรือฟอร์มสำคัญ เช่น ฟอร์มล็อกอิน, ฟอร์มลงทะเบียน, ฟอร์มที่มีไฟล์แนบ

\$_REQUEST: รวมค่าจากทั้ง GET, POST และ COOKIE ไว้ในตัวแปรเดียว ใช้งานง่ายแต่ไม่ควรใช้ในงานจริงเพราะไม่รู้แน่ชัดว่าค่าที่ได้มาจากช่องทางใด อาจทำให้เกิดช่องโหว่ด้านความปลอดภัยได้ง่ายกว่า

7.

Cookie คือข้อมูลขนาดเล็กที่เก็บไว้ที่เครื่องของผู้ใช้ (ฝั่ง browser) เซิร์ฟเวอร์เป็นผู้สั่งให้ browser เก็บ และ browser จะแนบ Cookie นั้นกลับมาทุกครั้งที่มี request ไปยังเซิร์ฟเวอร์เดิม

Session คือข้อมูลที่เก็บไว้ที่ฝั่งเซิร์ฟเวอร์ โดยเซิร์ฟเวอร์จะสร้าง Session ID ที่ไม่ซ้ำกันให้แต่ละผู้ใช้ แล้วส่ง Session ID นั้นไปเก็บไว้ใน Cookie ของผู้ใช้เพียงค่าเดียว เมื่อผู้ใช้ request มาใหม่ เซิร์ฟเวอร์จะใช้ Session ID นี้ไปดึงข้อมูลจริงที่เก็บไว้บนเซิร์ฟเวอร์ ระบบล็อกอินนิยมใช้ Session เพราะข้อมูลสำคัญ (เช่น สถานะการล็อกอิน, สิทธิ์การใช้งาน) ถูกเก็บไว้บนเซิร์ฟเวอร์ซึ่งผู้ใช้ไม่สามารถแก้ไขได้โดยตรง ต่างจาก Cookie ที่ผู้ใช้สามารถเปิดดูหรือแก้ไขค่าในเครื่องตัวเองได้ จึงปลอดภัยกว่า

8.

```
<?php
$conn = mysqli_connect("localhost", "root", "", "school_db");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

อธิบายทีละบรรทัด: `mysqli_connect(host, user, password, database)` พยายามเชื่อมต่อฐานข้อมูล และคืนค่าเป็น connection object หากสำเร็จ หรือคืนค่า `false` หากล้มเหลว บรรทัด `if (!$conn)` จึงใช้ตรวจสอบว่าเชื่อมต่อไม่สำเร็จหรือไม่ ถ้าไม่สำเร็จให้เรียก `die()` เพื่อหยุดสคริปต์พร้อมแสดงข้อความ error จาก `mysqli_connect_error()`

9.

```
<?php
for ($i = 2; $i <= 20; $i += 2) {
    echo $i . " ";
}
?>
```

`for` loop มีโครงสร้าง 3 ส่วนคือ (ค่าเริ่มต้น; เงื่อนไขที่ต้องเป็นจริงเพื่อวนต่อ; การเปลี่ยนค่าตัวแปรในแต่ละรอบ) ในที่นี้เริ่มที่ 2, วนจนกว่า \$i จะเกิน 20, และเพิ่มค่าทีละ 2 ทุกรอบ จึงได้เลขคู่ 2 4 6 ... 20 ตามที่โจทย์ต้องการ

10. ผลลัพธ์: index 1 = 7 index 3 = 9

`foreach ($arr as $i => $v)` จะวนอ่านทั้ง index (\$i) และค่า (\$v) ของแต่ละสมาชิกใน array `[3, 7, 1, 9]` คือ index 0=3, index 1=7, index 2=1, index 3=9 เงื่อนไข `if ($v > 5)` จะเป็นจริงเฉพาะค่า 7 (index 1) และ 9 (index 3) เท่านั้น ค่า 3 และ 1 ไม่ผ่านเงื่อนไขจึงไม่ถูก echo ออกมา

11.

ใช้ **Prepared Statement** (ผ่าน mysqli หรือ PDO) แทนการนำค่าที่ผู้ใช้กรอกไปต่อ (concatenate) เป็น string SQL โดยตรง เพราะการต่อ string ทำให้ผู้ไม่หวังดีสามารถแทรกคำสั่ง SQL ปลอมเข้าไปกับข้อมูลที่กรอกได้ (เช่น พิมพ์ `' OR '1'='1` ในช่อง password)

ตัวอย่างโค้ดที่ปลอดภัย:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
```

หลักการคือค่าที่ผู้ใช้กรอก (`$username`) จะถูกส่งแยกออกจากตัวคำสั่ง SQL ทำให้ฐานข้อมูลตีความว่าเป็นเพียง "ค่าข้อมูล" เท่านั้น ไม่สามารถกลายเป็นส่วนหนึ่งของคำสั่งได้ไม่ว่าผู้ใช้จะพิมพ์อะไรมาก็ตาม

12.

```
<?php
function sumTwo($a, $b) {
    return $a + $b;
}
echo sumTwo(3, 5); // แสดงผล 8
?>
```

การประกาศฟังก์ชันใน PHP ใช้ keyword `function` ตามด้วยชื่อฟังก์ชันและพารามิเตอร์ในวงเล็บ คำสั่ง `return` ใช้ส่งค่าผลลัพธ์กลับออกไปให้ผู้ใช้ฟังก์ชันนี้

13. ตอบ ข. ตรวจสอบว่าฟอร์มถูกส่งมาแบบ GET หรือ POST

มักใช้ร่วมกับ `if ($_SERVER['REQUEST_METHOD'] == 'POST') { ... }` เพื่อตรวจสอบว่าหน้านั้นถูกเปิดมาแบบปกติ (GET) หรือมีการ submit ฟอร์มเข้ามาแล้ว (POST) ก่อนจะเริ่มประมวลผลข้อมูลจากฟอร์ม เป็นรูปแบบที่ใช้บ่อยมากในไฟล์ PHP ที่ทั้งแสดงฟอร์มและรับข้อมูลอยู่ในไฟล์เดียวกัน

14.

class คือแม่แบบ (blueprint) ที่กำหนดว่าสิ่งที่จะสร้างขึ้นมามีคุณสมบัติและความสามารถอะไรบ้าง

object คือ instance หรือสิ่งที่ถูกสร้างขึ้นจริงจาก class นั้น (สร้างด้วย keyword `new`)

property คือตัวแปรที่อยู่ภายใน class ใช้เก็บคุณสมบัติของ object

method คือฟังก์ชันที่อยู่ภายใน class ใช้กำหนดพฤติกรรมของ object

ตัวอย่าง:

```
class Student {
    public $name;
    function greet() {
        return "Hi " . $this->name;
    }
}
$s = new Student();
$s->name = "Somchai";
echo $s->greet(); // Hi Somchai
```

สังเกตว่าภายใน method ต้องใช้ `$this->` เพื่ออ้างอิงถึง property ของ object ตัวเอง

15.

```
<?php
$username = $_POST['username'];
echo htmlspecialchars($username);
?>
```

`htmlspecialchars()` จะแปลงอักขระพิเศษของ HTML เช่น `<`, `>`, `"` ให้กลายเป็น entity ที่ปลอดภัย (เช่น `<`) ก่อนแสดงผล ทำให้ถ้าผู้ใช้พิมพ์ `<script>alert(1)</script>` เข้ามา มันจะถูกแสดงเป็นข้อความธรรมดาแทนที่จะถูก browser รันเป็นโค้ด (ป้องกัน Cross-Site Scripting หรือ XSS)

16. ตอบ ข. htmlspecialchars()

`strip_tags()` จะตัด tag HTML ออกไปทั้งหมด (ต่างจาก `htmlspecialchars` ที่แค่ทำให้ tag แสดงเป็นข้อความ ไม่ได้ตัดทิ้ง) ส่วน `trim()` ใช้ตัดช่องว่างหน้า-หลัง string และ `filter_input()` ใช้กรองค่าตามรูปแบบที่กำหนด (เช่น email, int) ซึ่งไม่ใช่หน้าที่หลักในการป้องกัน XSS โดยตรง

17.

Indexed Array: ใช้ตัวเลข 0,1,2,... เป็น key โดยอัตโนมัติ เช่น `$arr = ["a", "b", "c"];` เข้าถึงด้วย `$arr[0]`

Associative Array: ผู้เขียนกำหนด key เป็นข้อความเองเพื่อสื่อความหมาย เช่น `$student = ["name"=>"Somchai", "age"=>15];` เข้าถึงด้วย `$student['name']`

Associative Array เหมาะกับการเก็บข้อมูลที่มีหลายฟิลด์และต้องการให้อ่านโค้ดเข้าใจง่าย เช่น ข้อมูล 1 แถวจากฐานข้อมูล

18.

```
<?php
$page = $_POST['age'];
if (!is_numeric($age)) {
    echo "กรุณารอกตัวเลข";
}
?>
```

`is_numeric()` ตรวจสอบว่าค่าที่ส่งมาเป็นตัวเลข (รวมถึงทศนิยมและเลขติดลบ) หรือไม่ คืนค่า true/false เครื่องหมาย ! ด้านหน้าคือ "ไม่ใช่" ดังนั้นเงื่อนไขจะเป็นจริงเมื่อค่าที่กรอกไม่ใช่ตัวเลข จึงแสดงข้อความแจ้งเตือน

19. ตอบ ก. mysqli_close(\$conn)

ใช้ปิดการเชื่อมต่อฐานข้อมูลที่เปิดไว้ด้วย `mysqli_connect()` เพื่อคืนทรัพยากรของเซิร์ฟเวอร์ แม้ในทางปฏิบัติ PHP จะปิดการเชื่อมต่อให้อัตโนมัติเมื่อสคริปต์จบการทำงาน แต่การปิดเองอย่างชัดเจนเป็นแนวปฏิบัติที่ดี โดยเฉพาะสคริปต์ที่ทำงานนาน

20.

ขั้นตอนการทำงานของระบบ Login แบบทั่วไป:

- 1) ผู้ใช้กรอก username และ password ลงในฟอร์ม HTML แล้วกด submit (method="post")
- 2) ฟอร์มส่งค่าไปยังไฟล์ PHP ที่กำหนดใน action ของฟอร์ม
- 3) PHP รับค่าผ่าน `$_POST['username']` และ `$_POST['password']`
- 4) PHP ใช้ Prepared Statement query ฐานข้อมูลเพื่อค้นหา user ที่ตรงกับ username ที่กรอก
- 5) นำ password ที่ผู้ใช้กรอกไปเทียบกับ password ที่ hash เก็บไว้ในฐานข้อมูล โดยใช้ `password_verify($input, $hashInDb)`
- 6) หากตรงกัน ให้เรียก `session_start()` แล้วเก็บสถานะล็อกอิน เช่น `$_SESSION['user_id'] = $row['id'];` จากนั้น redirect ไปหน้าหลัก
- 7) หากไม่ตรงกัน ให้แสดงข้อความแจ้งเตือนว่า username หรือ password ไม่ถูกต้อง (ไม่ควรระบุว่าผิดที่ username หรือ password เพื่อความปลอดภัย)

1. ตอบ ข. **SELECT * FROM students**

SELECT คือคำสั่งดึงข้อมูล เครื่องหมาย ***** หมายถึง "ทุกคอลัมน์" ตามด้วย **FROM ชื่อตาราง** เพื่อระบุว่าดึงจากตารางไหน

2.

```
INSERT INTO students (name, age) VALUES ('สมชาย', 15);
```

โครงสร้างคือ **INSERT INTO ตาราง (คอลัมน์ที่จะใส่ค่า) VALUES (ค่าที่ตรงกันตามลำดับ)** ไม่จำเป็นต้องระบุคอลัมน์ id เพราะมักตั้งเป็น **AUTO_INCREMENT** ให้ระบบสร้างค่าให้เองอัตโนมัติ

3.

JOIN ใช้เพื่อรวมข้อมูลจากหลายตารางที่มีความสัมพันธ์กันผ่านคอลัมน์ที่อ้างอิงถึงกัน (Foreign Key)

INNER JOIN: ดึงเฉพาะแถวที่มีข้อมูล "ตรงกัน" ในทั้งสองตารางเท่านั้น หากฝั่งใดฝั่งหนึ่งไม่มีคู่ตรงกัน แถวนั้นจะไม่ถูกแสดง

LEFT JOIN: ดึงข้อมูล "ทุกแถว" จากตารางด้านซ้ายเสมอ แม้จะไม่มีคู่ตรงกันในตารางด้านขวา โดยคอลัมน์จากตารางขวาที่ไม่มีข้อมูลจะแสดงเป็น **NULL** เหมาะกับกรณีที่ต้องการแสดงข้อมูลหลักครบทุกรายการ เช่น แสดงนักเรียนทุกคนแม้บางคนยังไม่มีคะแนน

4.

```
SELECT name FROM students WHERE age > 15 ORDER BY age DESC;
```

WHERE age > 15 กรองเฉพาะแถวที่อายุมากกว่า 15 ส่วน **ORDER BY age DESC** เรียงผลลัพธ์จากอายุมากไปน้อย (**DESC** = descending) หากต้องการเรียงน้อยไปมากใช้ **ASC**

5.

Primary Key: คอลัมน์ (หรือกลุ่มคอลัมน์) ที่ใช้ระบุแต่ละแถวในตารางอย่างไม่ซ้ำกัน ห้ามเป็นค่าว่างและห้ามซ้ำ ตารางหนึ่งมี Primary Key ได้เพียงชุดเดียว

Foreign Key: คอลัมน์ในตารางหนึ่งที่เก็บค่าซึ่งอ้างอิงไปยัง Primary Key ของอีกตาราง ใช้สร้างความสัมพันธ์ระหว่างตาราง เช่น คอลัมน์ **student_id** ในตาราง scores อ้างอิงไปยัง **id** ในตาราง students

6.

```
SELECT class, COUNT(*) FROM students GROUP BY class;
```

GROUP BY ใช้จัดกลุ่มแถวที่มีค่าคอลัมน์ที่ระบุเหมือนกันไว้ด้วยกัน แล้วมักใช้ร่วมกับฟังก์ชันรวม (aggregate function) เช่น **COUNT()**, **SUM()**, **AVG()** เพื่อคำนวณต่อกลุ่ม ตัวอย่างนี้จะนับจำนวนนักเรียนในแต่ละห้องเรียน

7.

```
DELETE FROM students WHERE id = 5;
```

ข้อควรระวังสำคัญ: ต้องมี **WHERE** เสมอ ไม่เช่นนั้นคำสั่งจะลบข้อมูลทุกแถวในตารางทันที

8.

```
UPDATE students SET name = 'สมหญิง' WHERE id = 3;
```

โครงสร้างคือ UPDATE ตาราง SET คอลัมน์ = ค่าใหม่ WHERE เงื่อนไข เช่นเดียวกับ DELETE ต้องมี WHERE ไม่เช่นนั้นจะอัปเดตข้อมูลทุกแถวในตาราง

9.

WHERE: กรองแถวข้อมูลก่อนทำการจัดกลุ่ม ใช้กับคอลัมน์ปกติ ไม่สามารถใช้กับผลลัพธ์ของฟังก์ชันรวมได้

HAVING: กรองข้อมูลหลังจาก GROUP BY แล้ว ใช้กรองผลลัพธ์ของฟังก์ชันรวม เช่น `SELECT class, COUNT(*) FROM students GROUP BY class HAVING COUNT(*) > 20;` (แสดงเฉพาะห้องที่มีนักเรียนมากกว่า 20 คน) ซึ่งเขียนแบบนี้ด้วย WHERE ไม่ได้

10. ตอบ ข. CREATE TABLE

ใช้สร้างตารางใหม่ในฐานข้อมูล ต้องระบุชื่อคอลัมน์และชนิดข้อมูลของแต่ละคอลัมน์ในวงเล็บถัดจากชื่อตาราง

11.

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    class VARCHAR(20)  
);
```

`AUTO_INCREMENT` ทำให้ค่า id เพิ่มขึ้นอัตโนมัติทุกครั้งที่เพิ่มแถวใหม่โดยไม่ต้องระบุค่าเอง `PRIMARY KEY` กำหนดให้คอลัมน์นั้นเป็นคีย์หลักของตาราง `VARCHAR(100)` หมายถึงข้อความความยาวไม่เกิน 100 ตัวอักษร

12.

UNIQUE constraint บังคับว่าค่าในคอลัมน์นั้นห้ามซ้ำกันในทุกแถว เช่น อีเมลหรือเลขบัตรประชาชน ใช้เมื่อข้อมูลนั้นไม่ใช่ Primary Key ของตาราง แต่ก็ต้องไม่ซ้ำกัน (ต่างจาก Primary Key ตรงที่ตารางมี UNIQUE ได้หลายคอลัมน์ และบางระบบยอมให้ค่า NULL ปรากฏได้)

13.

```
SELECT students.name, scores.subject, scores.score  
FROM students  
INNER JOIN scores ON students.id = scores.student_id;
```

`ON students.id = scores.student_id` คือเงื่อนไขการจับคู่ระหว่างสองตาราง โดยจับคู่แถวที่ id ของ students ตรงกับ student_id ของ scores ผลลัพธ์จะได้ชื่อนักเรียนพร้อมวิชาและคะแนนของแต่ละคน

14.

```
ALTER TABLE students ADD email VARCHAR(100);
```

ALTER TABLE ใช้แก้ไขโครงสร้างของตารางที่มีอยู่แล้วโดยไม่ต้องลบตารางเดิม เช่น เพิ่มคอลัมน์ (ADD), ลบคอลัมน์ (DROP COLUMN), หรือแก้ไขชนิดข้อมูล (MODIFY)

15.

CHAR(n): ความยาวคงที่เสมอ n ตัวอักษร หากข้อมูลสั้นกว่าที่กำหนด ระบบจะเติมช่องว่างจนครบ เหมาะกับข้อมูลที่มีความยาวแน่นอนตายตัว เช่น รหัสไปรษณีย์ (5 หลัก)

VARCHAR(n): ความยาวผันแปรได้ตามข้อมูลจริง (สูงสุดไม่เกิน n) ใช้พื้นที่จัดเก็บเท่าที่จำเป็นจริง ประหยัดกว่าเมื่อความยาวข้อมูลไม่แน่นอน เช่น ชื่อ-นามสกุล

16.

Normalization คือกระบวนการจัดโครงสร้างตารางในฐานข้อมูลเพื่อลดความซ้ำซ้อนของข้อมูลและป้องกันปัญหาที่เรียกว่า Anomaly เมื่อเพิ่ม/ลบ/แก้ไขข้อมูล เช่น ถ้าเก็บข้อมูลครูอยู่ในหลายแถวของตารางเดียว เมื่อครูเปลี่ยนชื่อจะต้องแก้ทุกแถว (Update Anomaly) การ normalize จะแยกข้อมูลครูออกเป็นอีกตารางแล้วเชื่อมด้วย Foreign Key แทน ทำให้แก้ไขที่เดียวจบ ฐานข้อมูลจึงถูกต้องและดูแลรักษาง่ายขึ้น

17.

```
SELECT class, COUNT(*) AS total FROM students GROUP BY class;
```

AS total ใช้ตั้งชื่อคอลัมน์ผลลัพธ์ให้อ่านง่ายขึ้น (alias) ผลลัพธ์จะแสดงชื่อห้องพร้อมจำนวนนักเรียนในห้องนั้น

18. ตอบ ก. NOT NULL

เมื่อกำหนด **NOT NULL** ให้คอลัมน์ใด ฐานข้อมูลจะปฏิเสธการบันทึกข้อมูลทันทีหากคอลัมน์นั้นไม่มีค่า มักใช้กับฟิลด์ที่จำเป็นต้องมีข้อมูลเสมอ เช่น ชื่อ, username

เฉลยหมวด HTML Form

1. ตอบ ข. <form>

tag **<form>** เป็น container หลักที่ครอบ input ต่างๆ ไว้ทั้งหมด เมื่อกดปุ่ม submit ข้อมูลใน input ทุกตัวที่อยู่ภายใน form นี้ จะถูกส่งไปพร้อมกัน

2.

action: ระบุ URL หรือไฟล์ปลายทางที่จะรับข้อมูลฟอร์มไปประมวลผล เช่น **action="save.php"** หากไม่ระบุ ฟอร์มจะส่งไปยังหน้าเดิม

method: ระบุวิธีการส่งข้อมูล มีสองค่าหลักคือ **get** (ส่งผ่าน URL) และ **post** (ส่งซ่อนใน body ของ request)

3. ตอบ ค. password

<input type="password"> จะแสดงตัวอักษรที่พิมพ์เป็นจุดหรือดาว (•••) แทนตัวอักษรจริงบนหน้าจอ แต่ค่าที่ส่งไปจริงยังคงเป็นข้อความที่พิมพ์ (ไม่ได้เข้ารหัสใดๆ ในตัว HTML เอง ต้องอาศัย HTTPS และการ hash ฝั่ง server เพื่อความปลอดภัยจริง)

4.

```
<form action="save.php" method="post">
  <input type="text" name="fullname" placeholder="ชื่อ-นามสกุล" required>
  <input type="email" name="email" placeholder="อีเมล" required>
  <button type="submit">บันทึก</button>
</form>
```

attribute `name` ของแต่ละ input สำคัญมาก เพราะฝั่ง PHP จะใช้ชื่อนี้ในการดึงค่า เช่น `$_POST['fullname']` — ถ้าลืมใส่ `name` ค่าจะไม่ถูกส่งไปเลย `type="email"` ช่วยให้ browser ตรวจสอบรูปแบบอีเมลเบื้องต้นก่อน submit

5.

```
<select name="grade">
  <option value="m1">ม.1</option>
  <option value="m2">ม.2</option>
  <option value="m3">ม.3</option>
  <option value="m4">ม.4</option>
  <option value="m5">ม.5</option>
  <option value="m6">ม.6</option>
</select>
```

`<select>` สร้างกล่อง dropdown ส่วน `<option>` แต่ละตัวคือรายการตัวเลือกภายใน attribute `value` คือค่าที่จะถูกส่งไปจริงเมื่อ submit (ซึ่งอาจต่างจากข้อความที่ผู้ใช้เห็น)

6. ตอบ ข. บังคับให้ผู้ใช้กรอกข้อมูลก่อน submit

เป็นการ validate เบื้องต้นฝั่ง client (HTML5 validation) หาก input ที่มี `required` วางอยู่ browser จะไม่ยอมให้ submit และแสดงข้อความเตือน แต่ยังต้อง validate ข้างฝั่ง server (PHP) เสมอ เพราะ client-side validation สามารถถูกข้ามได้

7.

`<textarea>` ใช้รับข้อความหลายบรรทัด ปรับขนาดพื้นที่พิมพ์ได้ (rows, cols) เหมาะกับข้อมูลยาว เช่น ที่อยู่ หรือข้อความแสดงความคิดเห็น

`<input type="text">` รับข้อความได้เพียงบรรทัดเดียว เหมาะกับข้อมูลสั้น เช่น ชื่อ, อีเมล

8.

checkbox: เลือกได้หลายตัวเลือกพร้อมกัน อิสระจากกัน เหมาะกับสถานการณ์เช่น เลือกงานอดิเรกได้หลายอย่าง หรือยอมรับเงื่อนไขหลายข้อ

radio button: เลือกได้เพียงตัวเลือกเดียวในกลุ่มเดียวกัน (input ที่มี attribute `name` เหมือนกัน) เหมาะกับตัวเลือกที่ exclusive กัน เช่น เพศ, ระดับการศึกษาสูงสุด

9.

attribute `for` ของ `<label>` ต้องมีค่าตรงกับ `id` ของ input ที่ต้องการอธิบาย เช่น `<label for="uname">ชื่อผู้ใช้</label> <input id="uname" name="username">`

สำคัญด้าน accessibility เพราะ: (1) ผู้ใช้คลิกที่ข้อความ label ก็สามารถ focus ไปที่ input ได้ทันที ไม่ต้องเล็งคลิกตรงกล่องเล็ก ๆ

(มีประโยชน์มากกับ checkbox/radio) (2) โปรแกรมอ่านหน้าจอสำหรับผู้พิการทางสายตา (screen reader) จะอ่านชื่อ label ควบคู่กับ input ทำให้ผู้ใช้เข้าใจว่าต้องกรอกอะไร

10.

```
<div class="container">
  <form>
    <div class="form-group">
      <label for="name">ชื่อ</label>
      <input type="text" class="form-control" id="name" name="name">
    </div>
    <div class="form-group">
      <label for="age">อายุ</label>
      <input type="number" class="form-control" id="age" name="age">
    </div>
    <button type="submit" class="btn btn-primary">บันทึก</button>
  </form>
</div>
```

`container` จำกัดความกว้างของเนื้อหาให้เหมาะสมกับหน้าจอ, `form-group` จัดระยะห่างระหว่างแต่ละกลุ่ม label+input, `form-control` ทำให้ input มีความกว้างเต็มพื้นที่และมีสไตล์สวยงามตาม Bootstrap โดยอัตโนมัติ ทำให้ไม่ต้องเขียน CSS เอง และได้ผลลัพธ์ที่ responsive ในตัว

11. ตอน ข. ใน URL

เช่น หากฟอร์มมี input name="q" ค่า "php" แล้ว submit แบบ GET ไปที่ search.php จะเห็น URL เป็น `search.php?q=php`

12.

เพราะข้อมูลที่ส่งแบบ GET จะปรากฏอยู่ใน URL อย่างชัดเจน ทำให้ข้อมูลอ่อนไหว เช่น รหัสผ่าน มีความเสี่ยงรั่วไหลได้จากหลายช่องทาง เช่น (1) ประวัติการเข้าเว็บ (browser history) ที่บันทึกไว้บนเครื่อง (2) log ของเซิร์ฟเวอร์ที่มักบันทึก URL ที่เรียกเข้ามาทุกครั้ง (3) การแชร์ลิงก์โดยไม่ตั้งใจ จึงควรใช้ `method="post"` กับฟอร์มที่มีข้อมูลสำคัญเสมอ